

CS456 Design and Analysis of Algorithms

Project 2 - Enhancing Quicksort

Deepshan Adhikari (800846035)
deepadh@siue.edu

1. Introduction

The sorting of large datasets is one of the most basic problems in computer science. Quicksort is one of the most widely used sorting algorithms because it is fast and simple in practice. However, its speed depends heavily on how the pivot is chosen. If the pivot is chosen poorly, the algorithm slows down and can even become as slow as $O(n^2)$. If the pivot is chosen well, the algorithm stays fast at $O(n \log n)$.

1.1 Strategies Implemented

The goal of this project is to test four different ways of choosing a pivot in the Quicksort algorithm. Each way is tested on two kinds of arrays: random arrays and already sorted arrays. The arrays are also tested at different sizes to see how each method behaves as the input grows. To achieve this goal, four strategies for picking a pivot in Quicksort were implemented.

- a. **Default Quicksort:** This method picks the last element of the array as the pivot. It is the most common version and serves as a baseline for comparison.
- b. **Center Half Pivot:** This is the modified Quicksort from page 732 of Kleinberg and Tardos [1]. This method keeps picking random pivots until it finds one that sits in the center of the array. The center means the middle 50 percent of the sorted order. At least a quarter of the elements are smaller than the pivot and at least a quarter are larger. This makes sure the array splits into two fairly even halves each time.
- c. **Random Pivot:** This method picks any random element as the pivot. It does not check if the pivot is a good choice or not.
- d. **Quickselect Pivot:** This method runs the Quickselect algorithm at every step to find the exact median and uses that as the pivot. This guarantees a perfect split every time.

1.2 Theoretical Time Complexity

The Default Quicksort and Random Pivot both run in $O(n \log n)$ time on average. However, in the worst case, they slow down to $O(n^2)$. This happens when the pivot is always the smallest or largest element, which is exactly what happens on an already sorted array. Center Half Pivot and Quickselect Pivot both guarantee $O(n \log n)$ in all cases because they always pick a pivot that splits the array evenly. Center Half does this by rejecting pivots that are not in the middle 50 percent. Quickselect does this by finding the exact median every time.

The table below summarizes the time complexity of each strategy.

Strategy	Best Case	Average Case	Worst Case
Default Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Center Half Pivot	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$ expected
Random Pivot	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ with low probability
Quickselect Pivot	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$ expected

According to Kleinberg and Tardos [1], Section 13.5, the randomized divide and conquer approach gives expected $O(n)$ time for finding the median and expected $O(n \log n)$ time for Quicksort.

1.3 Implementation Details

The code was written and compiled in C++. Five array sizes were used: 10,000, 20,000, 50,000, 100,000, and 200,000 elements. For each size, two arrays were created:

- A random array with values between 1 and 1,000,000.
- A sorted array containing the values 1, 2, 3, ..., n in order.

All four sorting strategies were tested on both arrays at every size. Each test was run 10 times, and the average time was recorded. The timer started right after the array was ready and stopped as soon as the sort returned. Each sort was run in its own separate process, so that if the sort crashed, only that small process would die. The main program would notice the crash and print "Stack Overflow" instead of losing the whole run. Any single run that took longer than 10 minutes was printed as "too long".

1.4 Problems Encountered

Only one real problem came up during the experiment, Stack Overflow in the Default Quicksort on the sorted 200,000 array. On sorted input, the Default Quicksort always picks the last element as the pivot. The last element is always the biggest number, so every partition moves just one element out of the way and leaves the rest on one side. This makes the recursion go as deep as the array is long. For 200,000 elements, that means 200,000 nested function calls. Each call uses some stack memory, and once the total stack use crosses the operating system's limit, the process is killed by a signal.

This is not a bug in the benchmark. It is exactly the $O(n^2)$ worst case showing up physically. To handle it, each sort is run in a forked child process. When the child dies, the parent notices the signal and reports "Stack Overflow" for that cell instead of crashing the whole program.

2. Results

2.1 Running Times

The table below shows the average running time in milliseconds for each strategy on each array size. Each value is the average of 10 runs.

Array Size	Default Quicksort (random/sorted)	Center Half (random/sorted)	Random Pivot (random/sorted)	Quickselect Pivot (random/sorted)
10,000	2.9 / 193.4 ms	3.1 / 2.0 ms	1.5 / 1.2 ms	3.4 / 2.9 ms
20,000	2.2 / 773.5 ms	6.6 / 4.0 ms	3.0 / 2.3 ms	6.1 / 5.3 ms
50,000	5.3 / 4924.2 ms	18.0 / 9.7 ms	7.6 / 5.6 ms	17.5 / 14.5 ms
100,000	11.3 / 20183.2 ms	43.3 / 21.1 ms	16.1 / 11.8 ms	36.6 / 31.5 ms
200,000	26.0 ms / Stack Overflow	110.4 / 42.0 ms	33.5 / 24.1 ms	76.2 / 65.5 ms

At 200,000 elements, the Default Quicksort on the sorted array went to recursion too deeply and the operating system killed the process. This is the $O(n^2)$ worst case physically exceeding the stack limit, and the value is reported as Stack Overflow instead of a time.

2.2 Line Graphs

Quicksort All Strategies - Runtime vs. Array Size

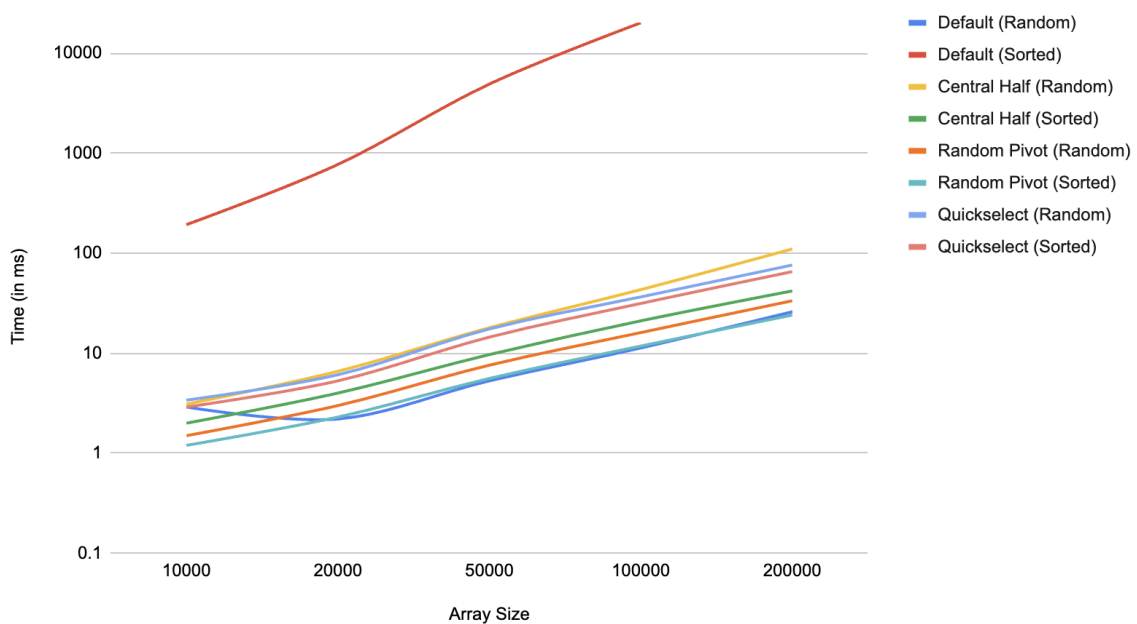


Figure 1: Running time of all four strategies on both input types (logarithmic scale).

The line graph above shows the running times from the table in section 2.1. The x-axis shows the array size and the y-axis shows the time in milliseconds. Both axes use a logarithmic scale so the differences between fast and slow strategies can be seen clearly. There are eight lines in total, one for each strategy on each input type.

All Strategies on Random Input - Runtime vs. Array Size

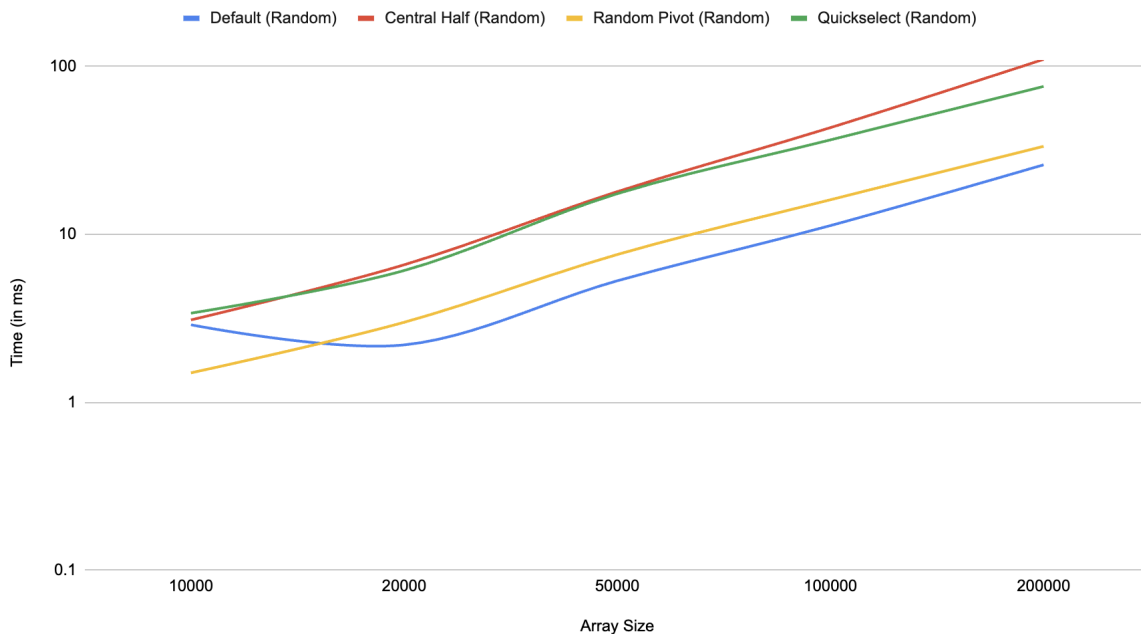


Figure 2: Running time of all four strategies on random input (logarithmic scale).

The line graph above shows only the random input results for all four strategies, using a logarithmic scale on both axes. The x-axis is the array size and the y-axis is the time in milliseconds. All four curves stay close together and grow slowly, which shows that when the input is random, the choice of pivot strategy has little effect on running time.

All Strategies on Sorted Input - Runtime vs. Array Size

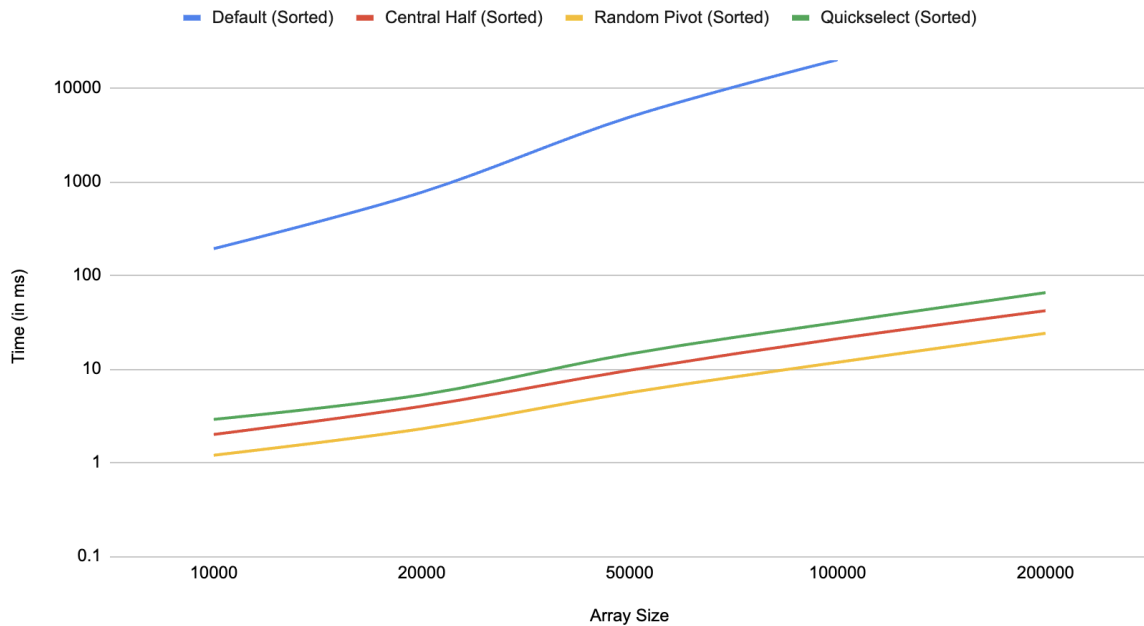


Figure 3: Running time of all four strategies on sorted input (logarithmic scale).

The line graph above shows the sorted input results for all four strategies, using a logarithmic scale on both axes. The x-axis is the array size and the y-axis is the time in milliseconds. Three of the four curves (Center Half, Random Pivot, and Quickselect) stay low and grow slowly, while the Default Quicksort curve rises steeply and leaves the chart range at the largest sizes. The Default curve stops at $n = 100,000$ because at $n = 200,000$ the recursion depth exceeded the operating system stack limit and the sort crashed. This makes the $O(n^2)$ growth of Default Quicksort on sorted input visible against the much slower growth of the other three strategies.

3. Discussion

3.1 Observed Speed Comparison Across Strategies

Looking at all the results together, three clear patterns can be seen.

First, on random input, all four strategies perform about the same. Default and Random Pivot are a little faster because they do less extra work. Center Half and Quickselect have extra overhead but it is not very large. At 200,000 elements, Default takes 26.0 ms, Random takes 33.5 ms, Quickselect takes 76.2 ms, and Center Half takes 110.4 ms.

Second, on sorted input the difference is dramatic. Default Quicksort slows down to $O(n^2)$ and becomes thousands of times slower as n grows. At 10,000 elements it took 193.4 ms, at 20,000 it took 773.5 ms, at 50,000 it took 4924.2 ms, and at 100,000 it took 20183.2 ms. At 200,000 elements it became so slow that the recursion exceeded the operating system stack limit and the process was killed. Center Half, Random Pivot, and Quickselect stayed fast on sorted input, finishing 200,000 elements in 42.0 ms, 24.1 ms, and 65.5 ms respectively.

Third, the growth rate is different. When the array size doubles, the balanced $O(n \log n)$ strategies take about twice as long, while Default Quicksort on sorted input takes about four times as long.

3.2 Actual vs. Theoretical Times

On random input, all four strategies grow like $O(n \log n)$. For example, Random Pivot grows from 1.5 ms at 10,000 elements to 33.5 ms at 200,000 elements. This is a factor of about 22 for a 20 times increase in size, which is in the expected range for $O(n \log n)$ once the small size effects are taken into account.

On sorted input, Default Quicksort shows clear $O(n^2)$ growth. Going from 10,000 to 20,000 elements (2 times size) increased the time from 193.4 ms to 773.5 ms, a factor of exactly 4. Going from 20,000 to 50,000 (2.5 times size), the time increased from 773.5 ms to 4924.2 ms, a factor of 6.4, which also almost matches $O(n^2)$. Going from 50,000 to 100,000 (2 times size), the time jumped from 4924.2 ms to 20183.2 ms, a factor of 4.1, again the signature of $O(n^2)$. The stack overflow at 200,000 is the result of a recursion depth of 200,000 function calls exceeding the operating system stack limit.

Center Half, Random Pivot, and Quickselect on sorted input all stayed near their random input times, which shows that a good pivot prevents the worst case. For example, at 200,000 elements, Random Pivot took 24.1 ms on sorted input and 33.5 ms on random input, which is close. Center Half took 42.0 ms on sorted and 110.4 ms on random.

The Quickselect overhead is visible on random data, 76.2 ms at 200,000 compared to 33.5 ms for Random Pivot, about 2.3 times slower. This matches the extra $O(n)$ median finding work at every level of recursion.

3.3 Surprising Observation

The most surprising result was that Center Half Pivot was consistently faster on sorted input than on random input at every size. At 200,000 elements it finished the sorted array in 42.0 ms but the random array in 110.4 ms. The same pattern holds at every size: 2.0 compared to 3.1 ms at 10,000; 9.7 compared to 18.0 ms at 50,000; 21.1 compared to 43.3 ms at 100,000.

The reason is that the Center Half validation loop accepts a pivot only if its rank lies in the middle half of the subarray. On a sorted array, almost any randomly selected element is already inside the middle half, so the loop succeeds on the first try almost every time. On a random array, the density of elements around any given candidate is more variable, so the loop sometimes has to retry, and each retry adds another full scan of the subarray. This shows how a scheme designed to protect against the sorted input worst case can actually do better on the very inputs it was meant to protect against.

3.4 Production Strategy Recommendation

Among the four strategies tested, Random Pivot Quicksort is the best choice for a production sorting library because of following reasons:

- It offers the best combination of speed and robustness.
- It avoids the $O(n^2)$ worst case of Default Quicksort on sorted input.
- It avoids the validation overhead of Center Half Pivot, which took 110.4 ms at 200,000 compared to Random Pivot's 33.5 ms.
- It avoids the extra $O(n)$ median finding cost per level of Quickselect Pivot, which took 76.2 ms at 200,000 compared to Random Pivot's 33.5 ms.
- Its expected running time is $O(n \log n)$ on every input, and its implementation is the simplest of the four.

In practice, production libraries often combine Randomized Quicksort with the median of three pivot selection, three way partitioning for inputs with many duplicates, and an insertion sort base case for small subarrays. That combination keeps the same $O(n \log n)$ expected behavior while reducing constant factors.

4. Code

```
// CS456 Design and Analysis of Algorithms
// Project 2 - Enhancing Quicksort

// Author: Deepshan Adhikari (800846035)
// deepadh@siue.edu

// This program benchmarks four pivot selection strategies for Quicksort:
// 1. Default Quicksort      - last element as pivot
// 2. Modified Quicksort    - retry until center half pivot
// 3. Random Splitter       - any random pivot
// 4. Quickselect at each step - median as pivot

// The benchmark tests five array sizes (10k, 20k, 50k, 100k, 200k) on two
// input types: random values in [1, 1,000,000] and sorted values 1..n
// Each combination is timed 10 times and the average is reported
// Any run over 10 minutes is reported as "too long"
// Any run whose child process is killed by a signal is reported as "Stack
// Overflow"

#include <iostream>    // console input and output (cout, endl)
#include <vector>      // dynamic array container (std::vector)
#include <random>       // random number generator (mt19937,
uniform_int_distribution)
#include <chrono>      // high-resolution timer (high_resolution_clock)
#include <iomanip>      // output formatting (setw, setprecision, fixed)
#include <sstream>      // string stream for building strings (stringstream)
#include <string>       // string type (std::string)
#include <algorithm>    // swap helper (std::swap)
#include <sys/wait.h>   // process utilities for fork/wait (waitpid, WIFSIGNALED)
#include <unistd.h>    // POSIX process utilities (fork, _exit)

using namespace std;
using namespace std::chrono;

// The random number generator. It is seeded from the operating system,
// so every run of the program produces different random arrays and
// different random pivot choices
static mt19937 g_rng(random_device{}());

// Any single run longer than this is reported as "too long"
static const double TIMEOUT_MS = 600000.0;

// Returns a uniformly random integer in the range [lo, hi]
```

```

static inline int randomInt(int lo, int hi)
{
    uniform_int_distribution<int> dist(lo, hi);
    return dist(g_rng);
}

// Partitions arr[low..high] around the pivot at pivotIdx
// Moves the pivot to the end, moves every element <= pivot to the front,
// and places the pivot in its final sorted position. Returns that index
static int partitionAroundPivot(vector<int> &arr, int low, int high, int pivotIdx)
{
    int pivotValue = arr[pivotIdx];
    swap(arr[pivotIdx], arr[high]); // move pivot to the end
    int i = low - 1;                // boundary of the "<=" region
    for (int j = low; j < high; ++j)
    {
        if (arr[j] <= pivotValue)
        {
            ++i;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]); // place pivot at its final index
    return i + 1;
}

// Default Quicksort.
// Always uses the last element of the current subarray as the pivot
// No depth limit and no fallback, so sorted input shows the O(n^2) case
static void defaultQuicksort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pi = partitionAroundPivot(arr, low, high, high);
        defaultQuicksort(arr, low, pi - 1);
        defaultQuicksort(arr, pi + 1, high);
    }
}

// Wrapper that sorts the whole array.
static void runDefault(vector<int> &arr)
{
    if (!arr.empty())
        defaultQuicksort(arr, 0, (int)arr.size() - 1);
}

```

```

// Returns true if the pivot at pivotIdx sits in the middle half of the
// subarray when sorted: at least 25% of elements are strictly smaller
// and at most 75% are smaller-or-equal
static bool isCenterHalf(const vector<int> &arr, int low, int high, int pivotIdx)
{
    int len = high - low + 1;
    int pivotValue = arr[pivotIdx];
    int less = 0, equal = 0;
    for (int i = low; i <= high; ++i)
    {
        if (arr[i] < pivotValue)
            ++less;
        else if (arr[i] == pivotValue)
            ++equal;
    }
    return (less >= len / 4) && (less + equal <= (3 * len) / 4);
}

// Picks random pivots and returns the first one that lies in the middle half
static int findCenterHalfPivot(const vector<int> &arr, int low, int high)
{
    for (int attempt = 0; attempt < 100; ++attempt)
    {
        int idx = randomInt(low, high);
        if (isCenterHalf(arr, low, high, idx))
            return idx;
    }
    return randomInt(low, high); // fallback after too many attempts
}

// Modified Quicksort (Center Half Pivot)
// Repeatedly chooses a random pivot until one lands in the middle half,
// which guarantees balanced partitions and O(n log n) expected time on
// every input, including sorted input
static void modifiedQuicksort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIdx = findCenterHalfPivot(arr, low, high);
        int pi = partitionAroundPivot(arr, low, high, pivotIdx);
        modifiedQuicksort(arr, low, pi - 1);
        modifiedQuicksort(arr, pi + 1, high);
    }
}

// Wrapper that sorts the whole array

```

```

static void runModified(vector<int> &arr)
{
    if (!arr.empty())
        modifiedQuicksort(arr, 0, (int)arr.size() - 1);
}

// Random Quicksort
// Picks any random element as the pivot, with no quality check
static void randomQuicksort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIdx = randomInt(low, high);
        int pi = partitionAroundPivot(arr, low, high, pivotIdx);
        randomQuicksort(arr, low, pi - 1);
        randomQuicksort(arr, pi + 1, high);
    }
}

// Wrapper that sorts the whole array.
static void runRandomSplitter(vector<int> &arr)
{
    if (!arr.empty())
        randomQuicksort(arr, 0, (int)arr.size() - 1);
}

// Quickselect Pivot
// Returns the index at which the k-th smallest element of arr[low..high]
// ends up, without fully sorting the range. At each step it picks a random
// pivot, partitions, and recurses only into the side that contains the k-th
// element
static int quickselectIndex(vector<int> &arr, int low, int high, int k)
{
    while (low < high)
    {
        int pivotIdx = randomInt(low, high);
        int pi = partitionAroundPivot(arr, low, high, pivotIdx);
        if (pi == k)
            return pi; // k-th element is at the pivot index
        else if (pi < k)
            low = pi + 1; // k-th element is on the right
        else
            high = pi - 1; // k-th element is on the left
    }
    return low;
}

```

```

// Quickselect Pivot
// Uses Quickselect at every step to place the exact median at its final
// position, then uses that median as the pivot. This gives perfectly
// balanced partitions at the cost of an extra O(n) selection pass at every level
static void quickselectPivot(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int medianIdx = quickselectIndex(arr, low, high, (low + high) / 2);
        int pi = partitionAroundPivot(arr, low, high, medianIdx);
        quickselectPivot(arr, low, pi - 1);
        quickselectPivot(arr, pi + 1, high);
    }
}

// Wrapper that sorts the whole array.
static void runQuickselect(vector<int> &arr)
{
    if (!arr.empty())
        quickselectPivot(arr, 0, (int)arr.size() - 1);
}

// Creates an array of the given size filled with random integers in the range [1,
// 1,000,000]
static vector<int> makeRandomArray(int size)
{
    vector<int> arr(size);
    uniform_int_distribution<int> dist(1, 1000000);
    for (int i = 0; i < size; ++i)
        arr[i] = dist(g_rng);
    return arr;
}

// Creates an array of the given size containing the values 1, 2, 3, ..., size
static vector<int> makeSortedArray(int size)
{
    vector<int> arr(size);
    for (int i = 0; i < size; ++i)
        arr[i] = i + 1;
    return arr;
}

// Runs one sort on a copy of the array inside a forked child process
// The copy is made before the timer starts. Returns the elapsed time in ms
static double timeOneRun(void (*sortFunc)(vector<int> &),

```

```

        const vector<int> &base)
{
    vector<int> copy = base; // copy is made before the timer starts

    auto start = high_resolution_clock::now();

    pid_t pid = fork();
    if (pid == 0)
    {
        // Child process: run the sort, then exit immediately
        sortFunc(copy);
        _exit(0);
    }

    // Parent process: wait for the child to finish or be killed
    int status = 0;
    waitpid(pid, &status, 0);

    auto end = high_resolution_clock::now();
    double elapsedMs = duration<double, milli>(end - start).count();

    if (WIFSIGNALED(status))
        return -2.0; // child was killed
    if (elapsedMs > TIMEOUT_MS)
        return -1.0; // run exceeded 10 minutes
    return elapsedMs;
}

// Runs the sort 10 times and returns the average in milliseconds
// If any single run returns one of the sentinels (-1.0 or -2.0), that value is
// returned immediately
static double timeMethod(void (*sortFunc)(vector<int> &),
                        const vector<int> &base,
                        int runs)
{
    double total = 0.0;
    for (int r = 0; r < runs; ++r)
    {
        double elapsed = timeOneRun(sortFunc, base);
        if (elapsed < 0.0)
            return elapsed;
        total += elapsed;
    }
    return total / runs;
}

```

```

// Formats a timing value for the table:
//   -2.0   -> "Stack Overflow"
//   -1.0   -> "too long"
//   >= 0.0 -> the value in milliseconds
static string formatTime(double ms)
{
    if (ms == -2.0)
        return "Stack Overflow";
    if (ms < 0.0)
        return "too long";
    stringstream ss;
    ss << fixed << setprecision(1) << ms;
    return ss.str();
}

// Formats one table cell as "<random time> / <sorted time>"
static string cell(double randomMs, double sortedMs)
{
    return formatTime(randomMs) + " / " + formatTime(sortedMs);
}

int main()
{
    // The five array sizes that will be tested
    const vector<int> sizes = {10000, 20000, 50000, 100000, 200000};

    // Number of trials per method per array.
    const int RUNS = 10;

    // Print the header.
    cout << "\nQuicksort Benchmark\n";
    cout << "Values are averages over " << RUNS << " runs.\n";
    cout << "Runs over 10 minutes are reported as \"too long\".\n";
    cout << "Crashes inside a sort (e.g. stack overflow) are reported as "
        << "\"Stack Overflow\".\n\n";

    // Column widths for the console table
    const int SIZE_W = 12;
    const int COL_W = 26;

    // Top line of the table header (strategy names)
    cout << left
        << setw(SIZE_W) << "Array Size" << " | "
        << setw(COL_W) << "Default Quicksort" << " | "

```

```

        << setw(COL_W) << "Center Half Pivot" << " | "
        << setw(COL_W) << "Random Pivot" << " | "
        << setw(COL_W) << "Quickselect Pivot" << "\n";

// Second line of the header (input types).
cout << left
    << setw(SIZE_W) << "" << " | "
    << setw(COL_W) << "(random/sorted)" << " | "
    << setw(COL_W) << "(random/sorted)" << " | "
    << setw(COL_W) << "(random/sorted)" << " | "
    << setw(COL_W) << "(random/sorted)" << "\n";

// Separator line.
cout << string(SIZE_W + 3 + 4 * (COL_W + 3), '-') << "\n";

// Run the experiment once per array size
for (int n : sizes)
{
    // Build the two input arrays for this size.
    vector<int> randBase = makeRandomArray(n);
    vector<int> sortedBase = makeSortedArray(n);

    // Time every strategy on both input types.
    double dR = timeMethod(runDefault, randBase, RUNS);
    double dS = timeMethod(runDefault, sortedBase, RUNS);
    double mR = timeMethod(runModified, randBase, RUNS);
    double mS = timeMethod(runModified, sortedBase, RUNS);
    double rR = timeMethod(runRandomSplitter, randBase, RUNS);
    double rS = timeMethod(runRandomSplitter, sortedBase, RUNS);
    double qR = timeMethod(runQuickselect, randBase, RUNS);
    double qS = timeMethod(runQuickselect, sortedBase, RUNS);

    // Print the size with a comma for readability, e.g. 10,000
    string sizeStr = to_string(n);
    if (sizeStr.size() > 3)
        sizeStr.insert(sizeStr.size() - 3, ",");

    // Print this row immediately so progress is visible
    cout << left
        << setw(SIZE_W) << sizeStr << " | "
        << setw(COL_W) << cell(dR, dS) << " | "
        << setw(COL_W) << cell(mR, mS) << " | "
        << setw(COL_W) << cell(rR, rS) << " | "
        << setw(COL_W) << cell(qR, qS) << "\n";
}

```

```
return 0;  
}
```

5. Conclusion

This project compared four pivot selection strategies for Quicksort on random and sorted inputs at five array sizes: 10,000, 20,000, 50,000, 100,000, and 200,000 elements.

The results show that pivot choice has a major effect on running time. Default Quicksort was fastest on random input because it does no extra work to choose a pivot. On sorted input, however, it slowed to $O(n^2)$ growth and at 200,000 elements the recursion exceeded the operating system stack limit and the process was killed.

Center Half Pivot, Random Pivot, and Quickselect Pivot all avoided this worst case. Each stayed fast on both input types, and their measured times matched the expected $O(n \log n)$ behavior. Among the three, Random Pivot was the fastest, Center Half was slightly slower because of its validation step, and Quickselect Pivot was the slowest because finding the median at every level adds an extra $O(n)$ pass.

The measured growth rates matched theory. Default Quicksort on sorted input grew quadratically with input size, while the other three grew near $n \log n$.

Based on these results, Random Pivot Quicksort is the best choice for a general purpose sorting library. It is fast on random input, robust on sorted input, and simple to implement. For even stronger protection, production libraries often combine Randomized Quicksort with the median of three pivot selection and an insertion sort base case for small subarrays.

References

[1] Kleinberg, Jon & Tardos, Éva. (2006). Algorithm Design.